



M363 Unit 4

UNDERGRADUATE COMPUTING

Software engineering with objects



The case study: Part 1

Unit **4**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate

ISBN 978 0 7492 1610 8



CONTENTS

1	Introduction	5
2	The Unified Process	7
2.1	Summary of section	8
3	Initial problem statement: the hotel chain	9
4	Understanding the domain	10
4.1	Business rules	10
4.2	Business processes	11
4.3	Summary of section	15
5	Scope of the new system	16
5.1	Summary of section	18
6	Volere template	19
7	Detailed software requirements	21
7.1	Elaborating the use cases	21
7.2	Software requirements	25
7.3	Summary of section	32
8	Summary	33
	Reference	35
	Index	36

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit, we investigate the requirements for a reservations system for a chain of hotels. We start from an initial problem statement and work through a range of activities, to arrive at sufficiently detailed requirements to support further analysis and the design and implementation of the system. Our goal in this unit, then, is to arrive at a set of use cases for the proposed system, along with a set of software requirements.

In later units, we will continue with the development of the system, based on the work carried out here. The overall development process we use will be loosely based on the Unified Process (Jacobson et al., 1999), and we give a general introduction to this in Section 2.

An initial problem statement contains information on the current business processes of a hotel in the chain. To elaborate on this information, we need to understand the domain well and document the business rules and current business processes; we will be using activity diagrams for the business processes. Domain modelling allows us to check that we understand the way the current business works. This is particularly important if we start with a poor conception of how the hotel operates its business. Another reason that domain modelling is useful is that some of the business rules and processes will carry through to the new system; therefore we need to understand them to be able to implement them in the new system. In practice, of course, domain modelling would be done in consultation with the stakeholders. The same is true of eliciting the initial problem statement. Indeed, these tasks would probably be carried out iteratively and would perhaps be intertwined.

From the initial problem statement, and the domain modelling, we will draw up a use case diagram that defines the scope of the system to be developed. Again, in practice, this would involve eliciting information from the stakeholders to elaborate on any details that are unclear or unspecified.

Having scoped the system, we then record the user requirements using the Volere template. Once again, in practice, this would generally involve consultation with stakeholders. In fact, such consultation is likely to occur throughout development.

The next phase of the requirements process is to draw up the software requirements. This work will draw on the artefacts produced previously. We begin by providing detailed scenarios for each of our use cases. We then extract from the use cases the detailed functional software requirements, refining these to the stage where they have clear fit criteria. We also need to ensure that the non-functional requirements are similarly refined, and indicate the use cases and/or individual functional requirements to which each non-functional requirement applies. In principle, the resulting functional and non-functional software requirements can be added to the Volere template (introduced in *Unit 2*). We use the same headings as the ones in the template without duplicating the whole template. In a large development it would be sensible to use a tool to manage the requirements; this might also save on duplication when a requirement gets updated.

The use cases and software requirements we develop in this unit will be used later during further analysis and design, to indicate what functionality needs to be supported. The software requirements play a number of roles:

- ▶ providing extra details during design, where a step in a use case scenario needs clarification;

- ▶ drawing up system tests – here we exploit the fit criteria of the functional and non-functional requirements;
- ▶ indicating, from non-functional requirements, how suitable our choice of architecture is;
- ▶ checking the design and implementation elements that support each use case.

In this unit, we develop a case study and present an approach to a given problem based on a number of decisions that could have been made differently. We are not claiming that the approach here is the best possible, only that it illustrates a plausible, practical application of the material in the preceding three units.

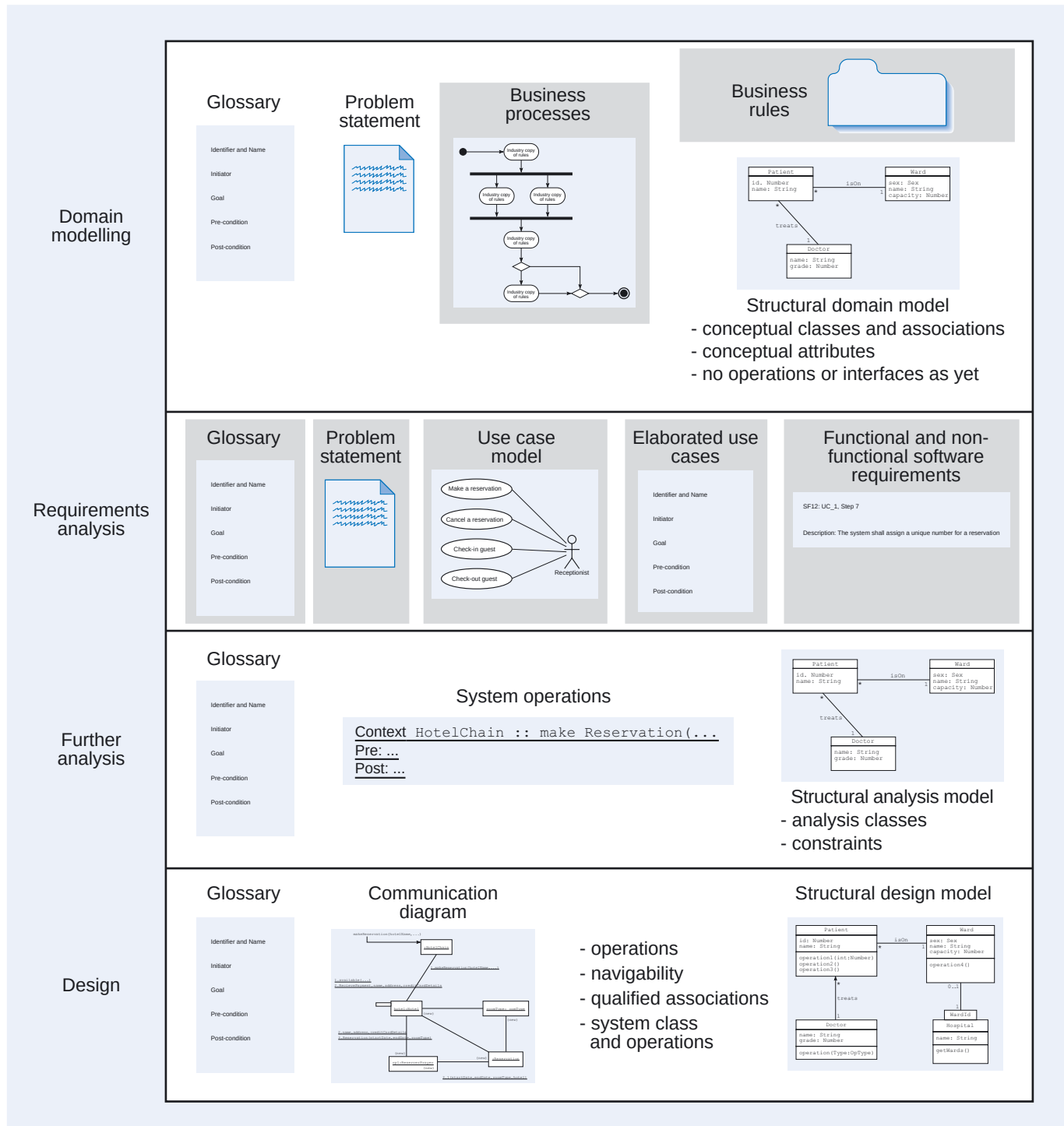


Figure 1 Parts of the development process covered in this unit

2

The Unified Process

The **Unified Process (UP)** (Jacobson et al., 1999) has emerged as a popular iterative and incremental development process for building enterprise systems based on an object-oriented approach. It promotes a set of best practices, namely that development should be organised in short, timeboxed iterations, and that it should be adaptive to accommodate inevitable change.

Timeboxing means that a (usually) short fixed period – for example, three to four weeks – is devoted to each iteration. Consequently, at each iteration, only a small set of requirements is considered and progressed to the implementation and testing stages. Each iteration results in a working, but possibly not yet complete, system that normally delivers an increment of functionality on the previous incomplete system. Typically, many iterations with progressive integration of increments are required before the product can be delivered.

Adaptive means that adjustments are allowed at each iteration. The motivation for this is the recognition that requirements may change throughout development, and that such changes should be welcomed rather than resisted. By involving customers and users at each iteration, feedback can be gained quickly, and the required adjustments made within the next iteration. Hence each iteration may provide an increment over, or simply revisit the output of, the previous one.

Other best practices promoted by UP are:

- ▶ dealing with high-risk issues in early iterations;
- ▶ prioritising the user's perspective by involving users in requirements, evaluation and feedback;
- ▶ building a view of the system's architecture in early iterations.

A UP project is organised into four major *phases*:

- 1 *Inception* – this is when the business case is developed, together with an idea of the scope of the system and a rough estimate of the effort required.
- 2 *Elaboration* – this is when the core of the system is developed in an iterative fashion. In this phase, all major risks are addressed and resolved; most of the requirements are identified and dealt with; and a more realistic estimate of the effort required is made.
- 3 *Construction* – this is when the final product is constructed including the remaining lower-risk and easier elements of the system, again in an iterative fashion.
- 4 *Transition* – this includes beta testing and deploying the system.

Within the UP phases, development work is organised within many *disciplines*.

'Discipline' is the UP term for a development activity such as requirements analysis or testing. An example of disciplines and their relationship to UP phases is shown in Figure 2.

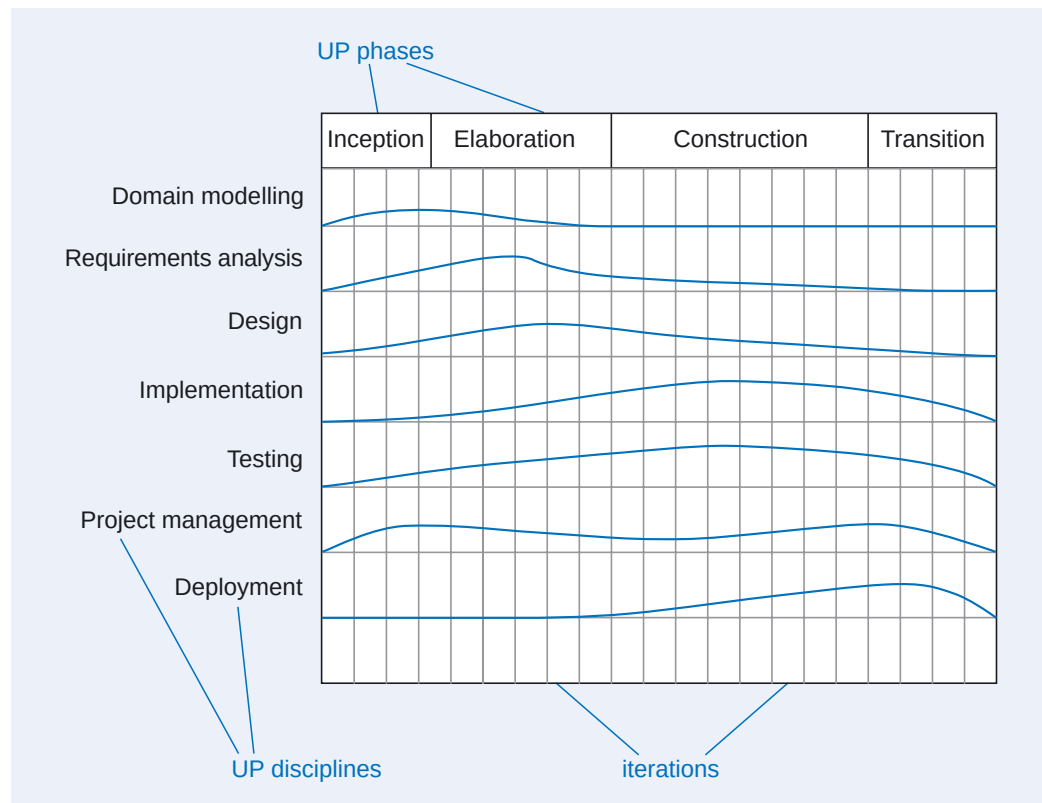


Figure 2 UP phases and disciplines

The figure is illustrative of what a UP project might look like. The columns represent iterations. For each discipline, the relative effort is represented throughout the UP phases and their iterations. For instance, most of the domain modelling occurs in the early iterations of the inception and elaboration phases, while most of the implementation occurs within the construction phase.

The approach we take to the case study is loosely based on the UP, with a focus on the elaboration phase and the disciplines of domain modelling (also referred to as **business modelling**), requirements analysis, design, implementation and testing.

The techniques used in the course are not specific to the UP, they can be used in many iterative and incremental processes based on an object-oriented approach to software.

We will focus on a single iteration of the system and will identify a small set of use cases as a suitable set of requirements for the first iteration. These will provide a usable slice of functionality. We will also focus during the first iteration on the main success scenarios for these use cases, although we will document relevant extensions for future reference.

2.1 Summary of section

The Unified Process is a popular iterative and incremental development process. It defines a series of timeboxed iterations and promotes a set of best practices: high-risk driven, user-centred and architecture-focused. The process followed in this unit will be loosely based on the Unified Process.

3

Initial problem statement: the hotel chain

A business that runs a chain of hotels in London has decided to commission a new computer system to manage reservations, and to check in and check out guests. Currently they have a software package that is able to carry out these tasks for an individual hotel. Each hotel runs this package. This is problematic, since, for example, if one hotel is full and receives a reservation request, the company tries to locate a suitable room in another of its London hotels; it is then necessary for a receptionist to ring around the other hotels. Similar problems arise if a customer wants to transfer between two hotels in the chain.

The new software should resolve this business problem by implementing a single, common reservation system across the whole chain. Additionally, the business wishes to gain a competitive edge with a frequent-guest programme to encourage repeat custom. Customers who rebook will be given discounts and other benefits, such as free newspapers.

The system should provide a system administrator with the ability to add new hotels to the system as they are acquired or built. Additionally, hotel managers should be able to add and remove rooms and room types to and from the system in response to activities such as decorating and building work.

Currently, each hotel deals only with reservation requests made by phone or email, but the future system must also deal with reservation requests made directly from a web browser. Payments are currently dealt with by each hotel, but in the new system, a web browser reservation will require the customer to enter details of a debit/credit card account, which will be debited automatically on the check out day. In the event of a late cancellation (less than 24 hours before the check in date) or a 'no show', the customer will automatically be charged for one night's accommodation. The total charges to all customers booking via the Web will be recorded on a weekly and monthly basis.

As the hotels do a lot of conference trade, block bookings are important to the business. A receptionist can check how many rooms are available in their hotel, but must then manually reserve each room required. Any overflow involves the receptionist phoning other hotels in the chain to book rooms elsewhere. This makes it hard to keep track of every person associated with a block booking. That in turn makes application of block-booking discounts extremely tedious. In the new system it must be possible to automate block bookings and deal with any overflow by automatically booking rooms in other nearby hotels in the chain.

4

Understanding the domain

We look now at the existing business, its processes, and the rules that constrain those processes. Understanding the current business situation is either the product of extensive domain expertise or the result of many observations of how people work, and discussions, consultations and interviews with domain experts, stakeholders and current users of the system.

4.1 Business rules

The rules of the existing business that have been identified by talking to domain experts so far are the following.

- ▶ A guest who cancels a reservation within 24 hours of the check in date will be charged for one night's accommodation; a 'no show' leads to the same charge.
- ▶ If a guest has reserved with a credit card and wants to pay with the same credit card, check out is speeded up.

Many other rules could be considered, such as a rule for what is done when there are overlapping reservations for the same guest. For the moment, we will keep a simple view of the existing system; later, we will deal with any more complex rules imposed on the new system, and how to model them. The current rules are important as they may still be imposed in the new system; decisions about which rules are relevant need to be taken in the elicitation of requirements for the new system. Here, we are particularly concerned with understanding how things are currently done and how we can model the activities that take place.

SAQ 1

Suggest a business rule that constrains overlapping reservations made by the same person.

ANSWER.....

Overlapping reservations made by the same person are allowed only if the guest names are different, or at least distinguished from one another.

4.2 Business processes

Below we describe the main processes of the current business. These are processes that will most likely still be carried out by the new system; it is therefore important to understand:

- ▶ the activities that currently take place;
- ▶ the problems currently encountered;
- ▶ what the users would like to do but are currently unable to do.

For each process, we give an informal description of its normal behaviour and describe the exchange of information between the different roles involved in that process.

Make reservation

The reservation process begins when someone contacts the hotel chain's reservation system, normally by phone. The caller will reserve a room in a particular hotel for one or more nights. The caller may make reservations on behalf of more than one person. Each hotel has more than one type of room. Hence callers must identify the type of room required by each potential guest, and the dates that they wish to stay. The existing system provides a price for each reservation of a given type of room, or suggests alternatives when there is no availability. If the caller accepts an offer, he or she must then provide the names and contact details for each guest identified in the reservation. Reservers need to provide credit card details (we use credit card, from now onwards, as a simplification for credit/debit card) to keep the reservation. The existing system confirms the reservation and provides a reservation number that identifies each reservation for a room. Potential guests and hotel staff can subsequently use the reservation number to speed up the process of checking in.

There are three roles in the reservation process: the reserver who initiates the process, the receptionist who deals with the request, and the existing hotel system. These are represented in Figure 3 by each of the three swimlanes. This activity diagram is only a first attempt at understanding the reservation process and more discussion is needed to get the exact detail.

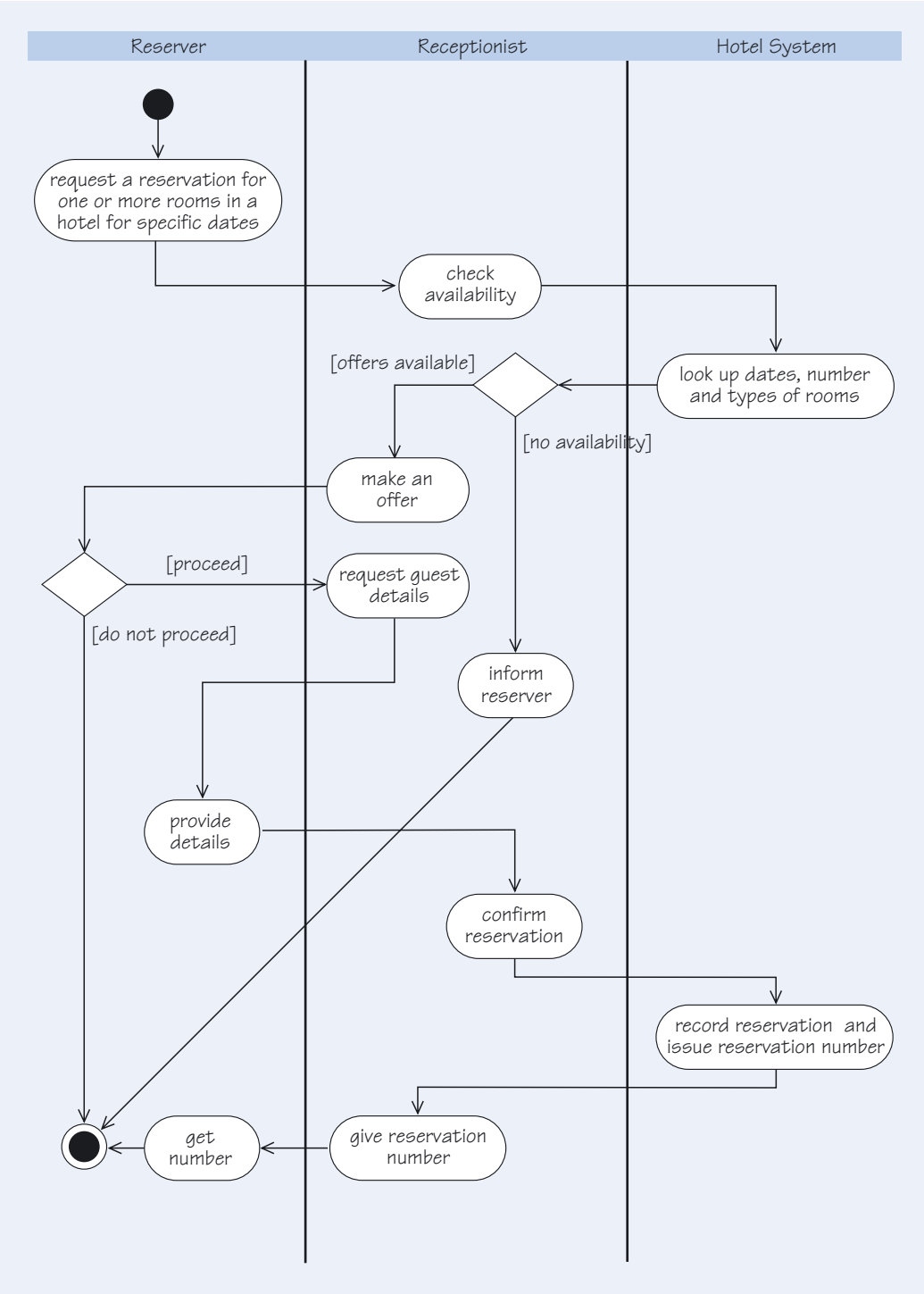


Figure 3 Activity diagram for the *make reservation* business process

Cancel reservation



Exercise 1

Write an informal description for the process that cancels a reservation, and draw the corresponding activity diagram.

Solution

Your solution may diverge from the one proposed here; this is just a first attempt at detailing the cancellation process.

Cancel reservation

The cancellation process begins when someone calls the hotel chain’s reservation system. The caller will cancel a reservation, giving the reservation number or the dates and name and address of the guest. The existing system looks up the reservation, checks whether the cancellation is being made less than 24 hours from the date of the reservation – in which case it charges the credit card with the amount corresponding to one night’s accommodation – closes the reservation and confirms the cancellation.

An activity diagram for this process is shown in Figure 4.

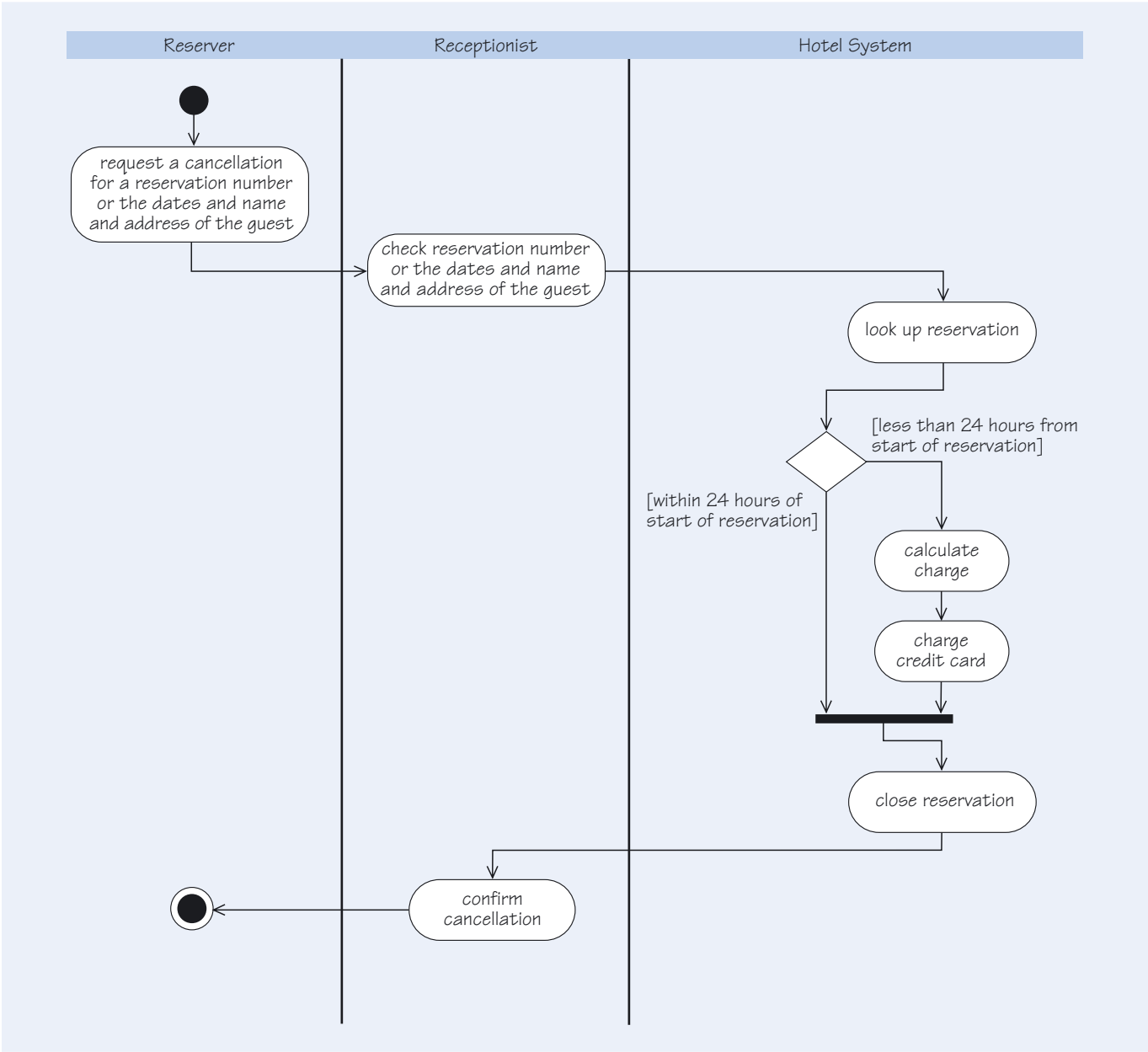


Figure 4 Activity diagram for the *cancel reservation* business process

Check in guest

Upon arrival, a guest gives the reservation number or the dates and name and address for his or her reservation to the hotel's receptionist, who enters it into the current system. The system reveals the details for that reservation so that the guest can confirm them. The software system allocates an appropriate room to that guest and opens up a bill for the duration of the stay. The receptionist issues a key for the room.

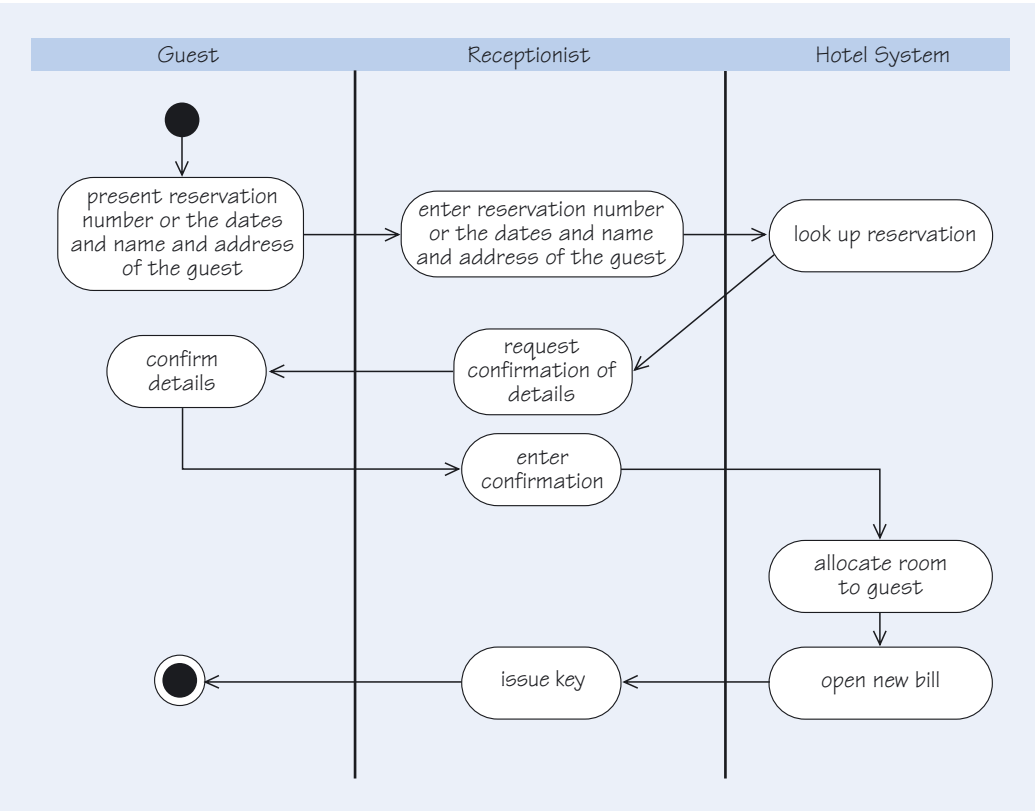


Figure 5 Activity diagram for the *check in guest* business process

In the check in process, instead of the reserver there is now the guest, that is, the person who is taking up the reservation. Note that the activity diagram of Figure 5 illustrates only the normal behaviour of the process, as we assume that the reservation number or the dates and name and address presented have a corresponding reservation. In understanding the current process, this is probably sufficient. When it comes to the new system to be developed, we need to detail how the system will react whenever the normal path is not followed.

Check out guest

Guests return their keys to the reception desk to signify that their stay in the hotel is over and they intend to leave. The software system uses the key number to identify and complete each guest's bill. Any other expenses made by the guest and not yet recorded in the system (such as the use of a mini-bar in the guest's room) are added. The system provides an itemised bill for confirmation by the guest. If the guest is paying with the credit card used for the reservation, then payment is processed immediately. Otherwise the guest has to pay, and once payment has been received from the guest, the system notifies the accounts system.

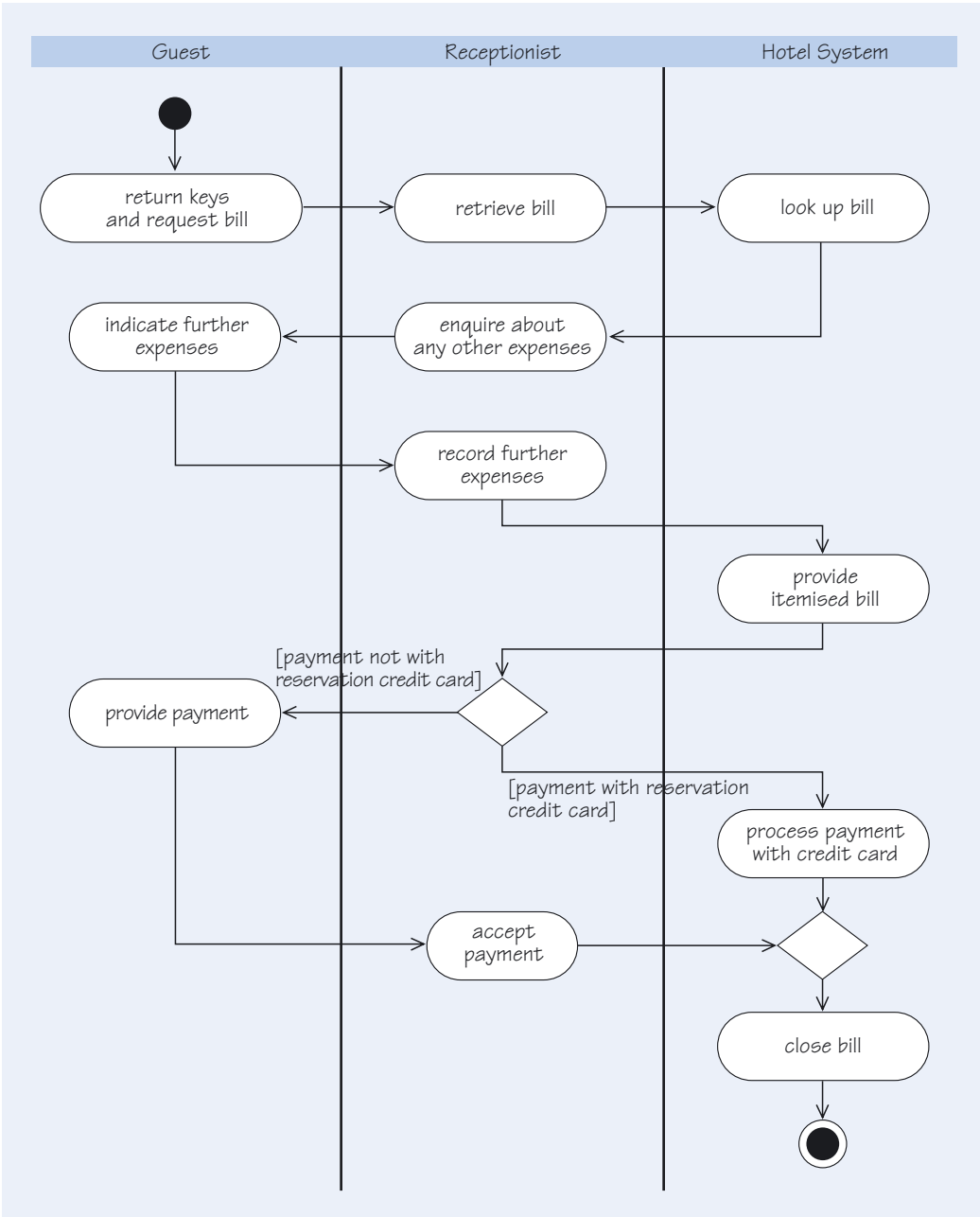


Figure 6 Activity diagram for the *check out guest* business process

4.3 Summary of section

In this section we described the current operation of the business, looking at its main processes and the rules that constrain them. The business processes need to be understood and any problems with them highlighted, before addressing a software solution.

5

Scope of the new system

Having understood the domain, and having an initial problem statement, we can now start thinking about the new system to be developed. In this section, we will determine the scope of the system from the user's perspective. In Section 6, we will then establish the user requirements, that is, what the proposed system should do from a user's perspective. The scope and requirements provide the basis for a contract between the customer and the developer.

The initial problem statement describes how the new system will be extended from a single hotel to deal with all the hotels owned. The system's interface will deal with all of the following:

- ▶ front desk staff in each hotel dealing with:
 - ▶ reservations by email, phone, post, or in person;
 - ▶ cancellations of reservations;
 - ▶ check in and check out of guests.
- ▶ anyone using a web browser to view information and make reservations;
- ▶ managers:
 - ▶ adding or removing a room type;
 - ▶ adding or removing a room.
- ▶ system administrators adding or removing a hotel;
- ▶ the external payment system to handle payments.

We will now look at the business processes identified for the existing system that will be carried through to the new system; each will correspond to a use case for the existing system.

- ▶ *make reservation* The receptionist interacts with the system to make a reservation.
- ▶ *cancel reservation* The receptionist interacts with the system to cancel a reservation.
- ▶ *check in guest* The receptionist interacts with the system to check in a guest.
- ▶ *check out guest* The receptionist interacts with the system to check out a guest.

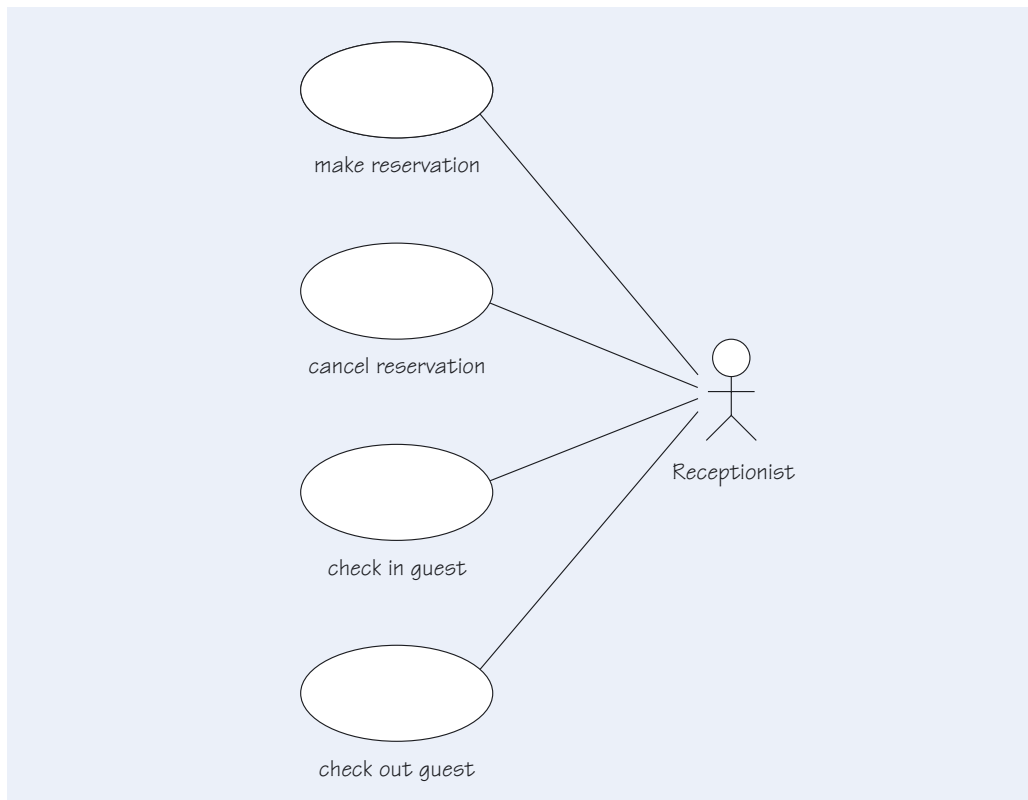


Figure 7 Use case diagram for the existing system

Note that in Figure 7, there is a single main actor: only the *Receptionist* interacts with the system. The new system will have other roles interacting with it.

The new system will deal with the following use cases.

- ▶ *add/remove hotel* A system administrator's role will include adding and removing hotels in the system.
- ▶ *add/remove room type* A manager's role will include adding and removing room types in the system.
- ▶ *add/remove room* A manager's role will include adding and removing rooms in the system.
- ▶ *make reservation* A reservation can be made either directly by a receptionist or on the internet by a reserver.
- ▶ *cancel reservation* A reservation can be cancelled either directly by a receptionist or on the internet by a reserver.
- ▶ *check in guest* A receptionist will check in guests.
- ▶ *check out guest* A receptionist will check out guests.

The new system is shown in Figure 8.

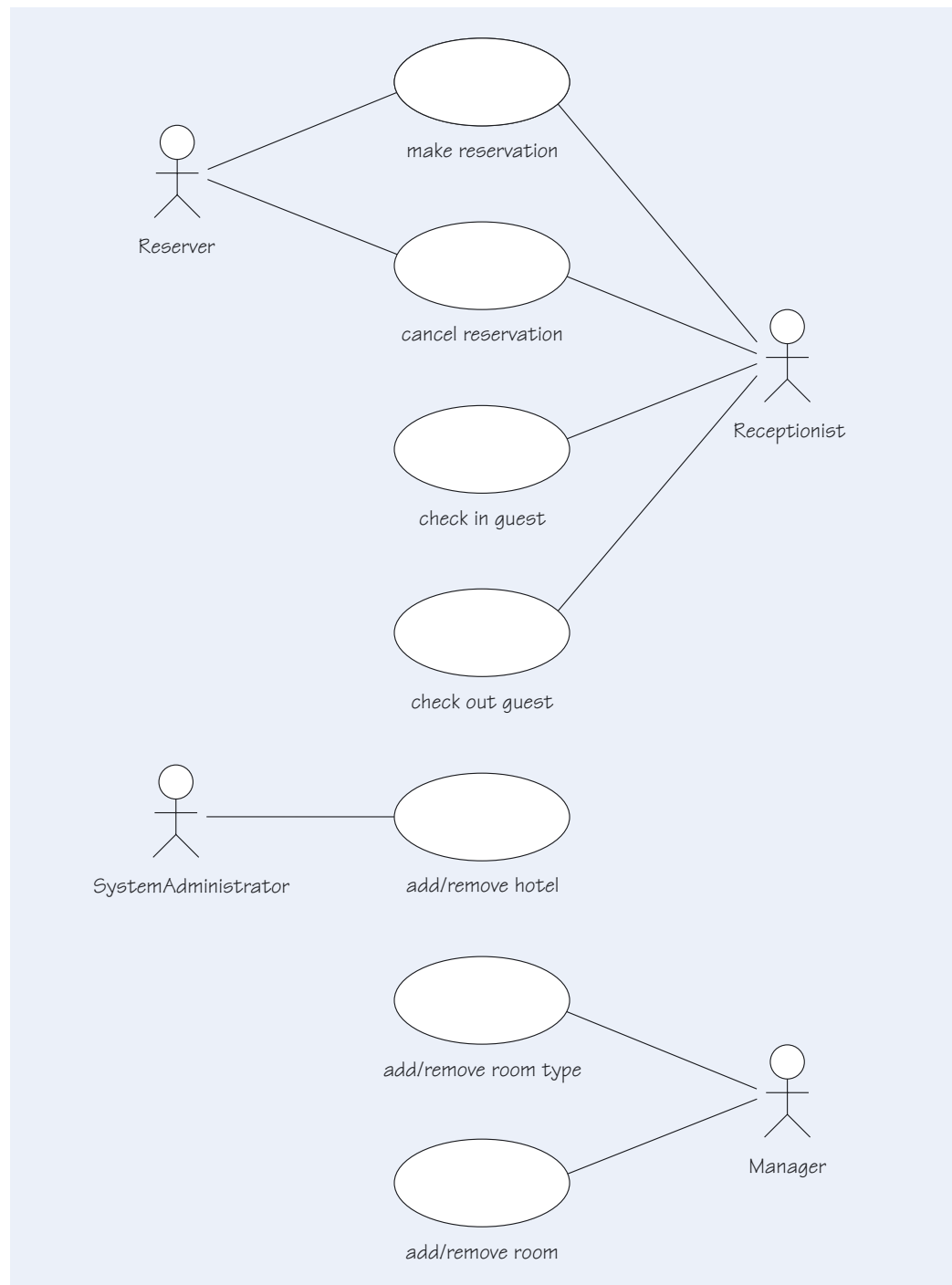


Figure 8 Use case diagram for the new system

We chose not to represent the payment system as it is not a main actor. So far, all we have identified are the main use cases of the system. Taking these together with the initial problem statement, we can now outline the main user requirements of the system.

5.1 Summary of section

The scope of the new system has been defined, listing the actors the system is to interact with and the use cases it should carry out.

- ▶ managers adding or removing a room type or a room;
- ▶ system administrators adding or removing a hotel;
- ▶ the external payment system to handle payments.

9 *Functional requirements.* The system shall:

- ▶ keep track of all hotels, room types and rooms;
- ▶ handle all reservations, cancellations and 'no shows';
- ▶ check in guests;
- ▶ check out guests;
- ▶ keep track of all current and past guests.

Non-functional requirements

10 *Look-and-feel requirements*

LF1: The system shall make use of a small number of bright colours, to conform to the brand image of the hotel chain.

LF2: The system shall have uncluttered forms.

11 *Usability requirements*

U1: The system shall be easy to use for receptionists, reservers using the internet, and managers.

12 *Performance requirements*

P1: The system shall be able to handle a range of large and small hotels.

P2: The system shall respond to most user input within 1 second.

P3: The system shall respond to a complex user request within 10 seconds.

P4: The system shall have high availability.

13 *Operational requirements*

O1: The system shall operate across the hotel chain with a set of terminals in each hotel and a single central server.

14 *Maintainability requirements*

M1: The system shall be able to add support for several European languages.

15 *Security requirements*

S1: Only managers shall be able to perform management operations.

S2: Credit/debit card details shall be securely managed.

S3: Reservers shall be able only to browse and make or cancel reservations.

S4: Only receptionists shall be able to check guests in or out.

S5: Information concerning which guests have been checked in or out shall not be alterable.

16 *Cultural and political requirements*

C1: The system shall reflect the child-friendly policy of the hotel chain.

17 *Legal requirements*

L1: The system shall operate in accordance with European and local law.

7

Detailed software requirements

Now we need to detail what the proposed software system must do, but avoid describing how it should work. If we do identify design options, we will record them for later discussion within the development team, and then go back and focus on the users' needs. Another risk is the introduction of new requirements by the developer. As you study the problem domain, it is possible that you will think of things that the new software system could or should do. By all means, discuss your ideas with the users, but do not invent requirements. In the case of the hotel chain, for example, you would formulate a series of questions for users such as the following.

- ▶ Does a reservation have to exist before someone can check in?
- ▶ What forms of payment are acceptable: cash, credit card and cheque?
- ▶ Is the information about a reservation to be kept after the payment? How long for?
- ▶ What information about guests is to be kept independently of having a current reservation?

Your aim is to construct a software system that will meet the needs of its users. Therefore you must identify 'who does what'. To do this, you will search or explore the problem description to determine the people involved and the work that they do. During that exploration, you are likely to identify a variety of people, some of whom may play a number of different roles. Your aim should be to find out who will really do the work that will be helped by using your software system. You should ask yourself who the actual users will be, and what tasks they must perform with the aid of your software system.

In the case study units, of which this is the first, we will deal only with a small set of use cases (*make reservation*, *cancel reservation*, *check in guest* and *check out guest*). Within this increment, we will look at just the first iteration through the use cases. This allows us to carry through the steps of development within our constraints (of time and space), and at the same time obtain quick feedback. Some issues, mainly related to non-functional requirements, will not be addressed in this iteration, as discussed later.

The first step we need to take is to elaborate the details of each use case assigned to iteration 1 in the form of a scenario expressed as a number of steps. From the scenario, we can extract the functional software requirements. We can then revisit the users' non-functional requirements in the Volere template, and refine them into software requirements as well.

7.1 Elaborating the use cases

We will elaborate each of the use cases identified for the first increment of the new system, using scenarios. For later reference, we will also give the names and identifiers of the other use cases.

Note that the *make reservation* use case can have either a reserver or a receptionist as its main actor. A reserver will trigger the use case when starting a reservation online; a receptionist will trigger the use case when requested to make a reservation by phone, by email or in person. Notice that the detail of the use case varies slightly from the domain process model in some respects.

The customer has indicated that:

- ▶ if the hotel requested has inadequate space, then it is acceptable to offer rooms in a nearby hotel within the chain;
- ▶ if a guest's details are already known, they should not be requested again.

We have also decided to tackle, in a first iteration, only the simplest case for a reservation: one room only, and the room will be allocated at reservation time.

Table 1 A textual description of the *make reservation* use case

Identifier and name	UC1 <i>make reservation</i>
Initiator	<i>Reserver or Receptionist</i>
Goal	A room in a hotel is reserved for a guest.
Precondition	None (that is, there are no conditions to be satisfied prior to carrying out this use case).
Postcondition	A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period.
Assumptions	The expected initiator is a reserver or receptionist, using a web browser to perform the use case. The guest is not already known to the hotel's software system (see step 5).
Main success scenario	
1	The reserver/receptionist makes a reservation request.
2	The reserver/receptionist selects the desired hotel, dates and type of room.
3	The hotel system provides the availability and price for the request; that is, one or more offers are made.
4	The reserver/receptionist accepts the offer.
5	The hotel system requests identification and contact details.
6	The reserver/receptionist provides identification and contact details of the guest for the hotel system's records, as well as credit card details.
7	The hotel system creates a reservation and gives it a number.
8	The hotel system allocates a room to the reservation.
9	The hotel system reveals the reservation number to the reserver/receptionist.
Extensions	
3.a.1	<i>No availability in this hotel.</i> The hotel system offers alternatives at a different hotel or hotels within a limited radius.
3.b.1	<i>No availability in this hotel or in any other hotels within a limited radius.</i> The hotel system informs the reserver and terminates the use case.
4.a.1	<i>Offer not accepted.</i> The hotel system terminates the use case.
6.a.1	<i>Guest already on record.</i> The scenario continues at step 7.

Exercise 2

Write a description of the *cancel reservation* use case.

Solution



Table 2 A textual description of the *cancel reservation* use case

Identifier and name	UC2 <i>cancel reservation</i>
Initiator	<i>Receptionist or Reserver</i>
Goal	A guest's reservation of a room at a hotel is cancelled.
Precondition	There is a reservation for a room at a hotel for a guest with a correct reservation number.
Postcondition	The reservation will have been cancelled, the allocated room will be freed, and a charge to the credit card will have been made if the cancellation was initiated within 24 hours of the start of the reservation.
Assumptions	The expected initiator is a receptionist or reserver.
Main success scenario	
1	The receptionist/reserver requests the cancellation of a reservation.
2	The receptionist/reserver enters the reservation number or the dates and name and address.
3	The hotel system retrieves the corresponding reservation.
4	The hotel system frees the allocated room.
5	The receptionist/reserver confirms the cancellation.
6	If the cancellation was initiated within 24 hours of the beginning of the reservation, the hotel system charges the credit card whose details are kept in the reservation.
7	The reservation is cancelled, and the hotel system informs the receptionist/reserver of the cancellation.
Extensions	
None	

Now we look at the *check in guest* use case.

Table 3 A textual description of the *check in guest* use case

Identifier and name	UC3 <i>check in guest</i>
Initiator	<i>Receptionist</i>
Goal	A guest takes up a reservation and occupies a room at the desired hotel.
Precondition	There is a room (of the desired type) reserved for the correct reservation number, and the guest can pay for the room.
Postcondition	The guest will have been allocated to a room for the period identified in the reservation, and a bill will have been opened for the duration of the stay.
Assumptions	The guest is already known to the hotel system. The hotel is confident that the guest can pay. For example, the guest has a valid credit card.
Main success scenario	
1	The guest provides a reservation number or the dates and name and address to the receptionist.
2	The receptionist enters the reservation number or the dates and name and address to find the reservation.
3	The hotel system retrieves the details of the requested reservation.
4	The receptionist confirms the details of the room type and duration of the stay with the guest.
5	The hotel system allocates the room to the guest.
6	The hotel system opens a bill for the guest. (It could be that there is a separate billing package, which must be notified upon check in.)
7	The receptionist issues a key to the guest.
Extensions	
None	

Finally, we look at the *check out guest* use case.

Table 4 A textual description of the *check out guest* use case

Identifier and name	UC4 <i>check out guest</i>
Initiator	<i>Receptionist</i>
Goal	A guest pays a bill and frees the hotel room.
Precondition	The guest currently occupies a hotel room.
Postcondition	The guest will have paid the bill for the room, and the room will have been freed.
Assumptions	The guest had provided details of a valid credit card, but may want to pay by different means.
Main success scenario	
1	The guest requires a bill for a room.
2	The receptionist enters the room number to find the bill.
3	The hotel system provides the details of the bill so far.
4	The receptionist requests details from the guest about any further expenses.
5	The guest provides details.
6	The receptionist calculates the final bill.
7	The receptionist processes payment with the credit card details held for the guest.
8	The hotel system closes the bill and frees the room.
Extensions	
7.a.1	<i>Guest wants to pay by different means.</i> Payment is provided.

The following are the identifiers and names of the other use cases:

UC5 *add hotel*

UC6 *remove hotel*

UC7 *add room type*

UC8 *remove room type*

UC9 *add room*

UC10 *remove room*

7.2 Software requirements

Our use cases show how actors will interact with the system. We will now draw up the software requirements showing what the system must do to support this. Each software requirement needs to be as unambiguous as possible. Note that we say *what* the system must do rather than *how* it will do it. Each requirement we arrive at will have a fit criterion attached; this adds precision and enables acceptance tests to be drawn up.

We will see that some of our functional requirements add relatively little information beyond that included in the corresponding scenario step, whereas in other cases there is more clarification. In a real system development, it would be sensible to apply some discretion as to whether all steps require elaboration into software requirements; we will see below that some steps are rather trivial.

Functional requirements: UC1 make reservation

We will begin by extracting initial software functional requirements from each of the steps in use case UC1, *make reservation*. Whenever there is an extension for a step, we write a separate corresponding functional requirement.

SFR1: UC1, Step 1

Description: The system shall allow a reserver/receptionist to initiate a reservation.

Fit criterion: The reservation page shall be displayed.

Some steps in the use case scenario will be decomposed into several system requirements, as is the case with Step 2 (SFR2–SFR4 below).

SFR2: UC1, Step 2

Description: The system shall obtain a hotel name from the user.

Fit criterion: A valid hotel name shall be accepted.

SFR3: UC1, Step 2

Description: The system shall accept a range of dates from the user.

Fit criterion: A valid range of dates shall be accepted.

SFR4: UC1, Step 2

Description: The system shall accept a room type from the user.

Fit criterion: A valid room type shall be accepted.

SFR5: UC1, Step 3

Description: The system shall check the availability of a given room type in a given hotel for a given range of dates. One or more offers shall be made, and their prices shall be stated.

Fit criterion: Offers consistent with the availability of a room shall be made, and the correct price for the room shall be supplied.

SFR5.a.1: UC1, Step 3.a.1

Description: *No availability in this hotel*. The system shall check the availability of alternatives at a different hotel or hotels within a limited radius. One or more offers shall be made, and their prices shall be stated.

Fit criterion: Offers consistent with the availability of a room shall be made, and the correct price for the room shall be supplied.

SFR5.b.1: UC1, Step 3.b.1

Description: *No availability in this hotel or in any other hotels within a limited radius*. The reserver shall be informed and the use case exited.

Fit criterion: No offer shall be made, and the reserver shall be informed.

SFR6: UC1, Step 4

Description: The system shall accept confirmation from the user.

Fit criterion: A Yes response shall be accepted.

SFR6.a.1: UC1, Step 4.a.1

Description: *Offer not accepted*. The system shall accept non-confirmation from the user and exit the use case.

Fit criterion: A No response shall be accepted.

SFR7: UC1, Step 5

Description: The system shall allow identification and contact details to be entered.

Fit criterion: The guest details page shall be displayed.

SFR8: UC1, Step 6

Description: The system shall accept the identification of the guest.

Fit criterion: Identification details shall be accepted.

SFR9: UC1, Step 6

Description: The system shall accept contact details of the guest.

Fit criterion: The street number, street name, town and postcode shall be accepted.

SFR9.a.1: UC1, Step 6.a.1

Description: *Guest already on record.* The system shall proceed to Step 7.

Fit criterion: The system shall proceed to Step 7 if guest identification is found.

SFR10: UC1, Step 6

Description: The system shall accept valid credit card details.

Fit criterion: A valid credit/debit card number shall be accepted.

SFR11: UC1, Step 7

Description: The system shall create a reservation in a given hotel for the given dates.

Fit criterion: A reservation shall be created for the guest between the given dates.

SFR12: UC1, Step 7

Description: The system shall assign a unique number for a reservation.

Fit criterion: A unique number shall be assigned to the reservation.

SFR13: UC1, Step 8

Description: The system shall allocate a room for a reservation.

Fit criterion: A room shall be assigned to the reservation.

SFR14: UC1, Step 9

Description: The user shall be issued with the reservation number.

Fit criterion: The correct reservation number shall be issued.

While we can see that some of these requirements add very little to the corresponding scenario steps, there are some important exceptions. For example, in SFR3, value is added by validation of the date. This may require strengthening an existing functional requirement.

Functional requirements: use cases 2, 3 and 4

Most of the steps in use cases 2, 3 and 4 are relatively straightforward. This is partly because a lot of the relevant information has been collected in use case 1 and so can be taken as a given. We have therefore decided not to explicitly state the software requirements for use cases 2, 3 and 4. We also feel that a designer who has worked with the software requirements for use case 1 will be in a position to interpret other use cases based on that experience. We would take a different view if we were developing a control system for some potentially dangerous machinery. This is the sort of pragmatic decision that can be made on the basis of experience. In a real development, it is important to use appropriate techniques (and tools) where they add value, and not simply because they can be used.

Non-functional requirements

SAQ 2

- (a) In the Volere template in Section 6, we recorded some non-functional requirements. What further work is needed in order to produce the software requirements?
 - (b) What else needs to be done in terms of non-functional requirements?
- ANSWER.....
- (a) We need to remove any ambiguities and add clear fit criteria. Additionally, we need to identify which functional requirements they constrain; in some instances, they will apply to whole use cases. Often we will need to consult the system stakeholders for further details.
 - (b) We should look at each of the functional software requirements in turn, and consider whether there are any non-functional requirements that apply to them but have been omitted. We should not assume that the stakeholders will supply a complete set of requirements, so cross-checks such as these are essential.

In Exercise 3, we consider the non-functional software requirements. Your solution will probably vary quite considerably from that given below, owing to its open nature. The solution is therefore indicative, but it will be used later on in the case study.



Exercise 3

- (a) Take the non-functional requirements recorded in the Volere template and state appropriate fit criteria, together with the use cases and, where appropriate, specific functional requirements to which they apply. Assume that the stakeholders have been consulted and have supplied appropriate information.
- (b) Consider each of the functional software requirements given above for UC1, and consider whether any of the non-functional requirements can be refined.

Solution.....

- (a) The following non-functional requirements are suggested.
 - Look-and-feel requirements*
 - LF1: The system shall make use of a small number of bright colours, to conform to the brand image of the hotel chain.
 - On consultation with the stakeholders we arrive at the following.*
 - Fit criterion: The system shall be designed with black and white forms with a sky blue title bar and a yellow motif in the bottom right-hand corner copied from the company logo.
 - Applies: All use cases.
 - LF2: The system shall have uncluttered forms.
 - Fit criterion: Each form shall be limited to a maximum of 8 textual input boxes; each box shall have an accompanying caption of up to 20 characters. Any preamble text shall be limited to 20 words.
 - Applies: All use cases.

Usability requirements

U1a: The system shall be easy for receptionists to use.

Fit criterion: Receptionists shall be able to learn how to use the system in 1 hour. All current receptionists shall be able to complete a reservation, check in or check out form in 2 minutes, assuming the guest has no special requirements.

Applies: UC1, UC2, UC3, UC4.

U2b: The system shall be easy for reservers to use.

Fit criterion: The system shall require no training for its online facilities. At least 90% of reservers shall be able to make a reservation in 1 minute.

Applies: UC1, UC2.

U3c: The system shall be easy for managers and system administrators to use.

Fit criterion: Managers and system administrators shall be able to learn how to use the system in 2 hours. All of the current managers shall be able to perform any of the management operations in 1 minute.

Applies: UC5, UC6, UC7, UC8, UC9, UC10.

Performance requirements

P1: The system shall be able to handle a range of large and small hotels.

Fit criterion: The system shall be able to handle up to 100 hotels, varying in size from 10 rooms to 1000 rooms.

Applies: All use cases.

P2: The system shall respond to most user inputs within 1 second.

Fit criterion: The system shall be able to respond to more than 50% of user inputs within 1 second.

Applies: All use cases, except UC1.

The exceptions in the above fit criterion relate to user requests where we expect the system to have to do a greater amount of processing than we can guarantee in 1 second.

P3: The system shall respond to a complex user request within 10 seconds.

Fit criterion: The system shall be able to respond to all complex user requests within 10 seconds.

Applies: UC1 (since the system needs to search for availability).

P4: The system shall have high availability.

Fit criterion: The system shall be available 99% of the time for 24 hours a day, 7 days a week, with any period of unavailability lasting 1 hour at most.

Applies: All use cases.

Operational requirements

O1: The system shall operate across the hotel chain, with a set of terminals in each hotel and a single central server.

Fit criterion: Self-contained.

Applies: All use cases.

Maintainability requirements

M1: The system shall be able to add support for several European languages.

Fit criterion: Another 2 European languages shall be supported in addition to English.

Applies: All use cases.

Security requirements

S1: Only managers and system administrators shall be able to perform management/administration operations.

Fit criterion: No more than one breach per year shall occur.

Applies: UC5, UC6, UC7, UC8, UC9, UC10.

S2: Credit/debit card details shall be securely managed.

Fit criterion: A fraud shall occur in no more than one in a million card transactions.

Applies: UC1, UC2, UC3, UC4 specifically, but also to any part of the system where credit card details are accessible.

S3: Reservers shall be able only to browse and make or cancel reservations.

Fit criterion: No more than one breach per year shall occur.

Applies: UC1, UC2.

S4: Only receptionists shall be able to check guests in or out.

Fit criterion: No more than one breach per year shall occur.

Applies: UC3, UC4, but also to any part of the system where relevant details are accessible.

S5: Information on which guests have been checked in and out should not be alterable.

Fit criterion: No more than one breach per year shall occur.

Applies: UC3, UC4, but also to any part of the system where relevant details are accessible.

Cultural and political requirements

C1: The system shall reflect the child-friendly policy of the hotel chain.

Fit criterion: The system shall distinguish between child and adult guests in order that children can be treated appropriately.

Applies: UC1, UC2, UC3, UC4.

Legal requirements

L1: The system shall operate in accordance with European and local law.

Fit criterion: The system shall pass an audit by the hotel chain's legal department.

Applies: All use cases.

- (b) Looking at the individual functional requirements for the use cases, it seems clear that only functional software requirements SFR5, SFR5.a.1, SFR7, SFR9, SFR10 and SFR13 can be considered as complex user requests. So it would seem reasonable to refine our performance requirements in two ways. Firstly, we will allow specific times for each of these steps. Secondly, we will allow 1 second for each of the other steps.

Now, the relevant performance requirements become as follows.

P2: The system shall respond to most user input within 1 second.

Fit criterion: The system shall be able to respond to more than 50% of user inputs within 1 second.

Applies: All use cases, except UC1 – SFR5, SFR5.a.1, SFR7, SFR9, SFR10, SFR13.

P3: The system shall respond to a complex user request within 10 seconds.

Fit criterion: The system shall be able to respond to more than 50% of user inputs within 10 seconds.

Applies: UC1 – SFR5, SFR5.a.1, SFR7, SFR9, SFR10, SFR13.

A similar process of refinement of these requirements would be appropriate in respect of the other use cases, so the information about where the requirements apply will continue to become more specific.

You may have found other specific details that require the given non-functional software requirements to be refined in this way. Alternatively, you may have found some completely new non-functional requirements.

Checking for completeness

Our software requirements have now become quite detailed, and at this point it is appropriate to check them for completeness against the user requirements and in particular the contents of the Volere template.

For example, comparing use case UC1 against the purpose of the product in the template, we find that we have not addressed the frequent-guest programme. We need to either elaborate on our existing functional requirements or add further ones. We use the former approach here. We need to amend the way prices are calculated, and associate repeat-custom information with the guest. On consultation with the stakeholders, we also learn that it would be useful for reservations from repeat guests to have this indicated clearly in the reservation code. In fact, the stakeholders thought we knew this and were expecting it in the system. We therefore amend a further software requirement, and note that we should add information about the reservation code to the user requirements in the original Volere template.

The updated software requirements are as follows.

SFR5: UC1, Step 3

Description: The system shall check the availability of a given room type in a given hotel for a given range of dates. One or more offers shall be made, and their prices shall be stated. The price shall take account of the frequent-guest programme.

Fit criterion: Offers consistent with the availability of a room shall be made, and the correct price for the room shall be supplied, taking account of the frequent-guest programme.

SFR5.a.1: UC1, Step 3.a.1

Description: *No availability in this hotel.* The system shall check the availability of alternatives at a different hotel or hotels within a limited radius. One or more offers shall be made, and their prices shall be stated, taking account of the frequent-guest programme.

Fit criterion: Offers consistent with the availability of a room shall be made, and a correct price for the room shall be supplied, taking account of the frequent-guest programme.

SFR12: UC1, Step 7

Description: The system shall assign a unique number for a reservation. If the guest has stayed in one of the hotels before, the reservation code shall begin with an R.

Fit criterion: A unique number shall be assigned to the reservation. This number shall be preceded by an R if the guest has stayed in one of the chain's hotels before.

In order to keep our first iteration focused, we will use the original requirements in the rest of the case study.

SAQ 3

- (a) Is it also necessary to check the non-functional software requirements against the original user requirements?
- (b) Is it possible that elaborating the functional requirements as above might require us to revise our non-functional requirements? What if we were to add new functional requirements?
- (c) Can elaboration of our software requirements lead to inconsistencies? Give an example.

ANSWER.....

- (a) Yes. For example, as well as simply missing explicit non-functional requirements in the template, there might well be non-functional requirements implied under product constraints.
 - (b) Yes, in both cases. There may well be constraints on the added functionality that are not covered by the existing non-functional requirements.
 - (c) Yes. For example, if we add sufficient extra functionality, the time taken by the system to carry out the necessary work might mean we are unable to satisfy the time constraint in usability requirement U1.
-

7.3 Summary of section

In this section we detailed the software requirements for the software system to be developed. We started by describing use cases in detail; from the scenarios we extracted the functional software requirements; we then refined the users' non-functional requirements in the Volere template with fit criteria and checked for completeness.

8

Summary

This is a practical unit, in which you have had a chance to apply the concepts learnt so far about software development.

We have introduced you to the Unified Process and its main characteristics, some of which will be present in the development followed in the case study units.

In this unit, you have seen how to carry out domain analysis to understand a real-world situation, identifying the main business processes and representing them. You have decided on the scope of a software system to be developed to address an initial set of user requirements, and elaborated on some of the main use cases for such a system. You have seen how to document the initial set of requirements using the Volere template, and how to elicit detailed software functional and non-functional requirements and their fit criteria.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ explain the main characteristics of the Unified Process;
- ▶ analyse the description of a domain situation, and identify business rules and business processes;
- ▶ represent processes with activity diagrams;
- ▶ define the scope for a software system, and identify and describe its main use cases;
- ▶ outline a requirements document based on the Volere template;
- ▶ elicit software functional requirements from use cases and scenarios, and define their fit criteria;
- ▶ elaborate on non-functional requirements and their fit criteria.

Reference

Jacobson, I., Booch, G. and Rumbaugh, J. (1999) *The Unified Software Development Process*, Reading, MA, Addison-Wesley.

Index

A

adaptability 7

B

business modelling 8

business processes 10

business rules 5

C

completeness 31

construction phase of a UP project 7

current business processes 5

D

design 8

disciplines 7

domain modelling 8

E

elaboration phase of a UP project 7

I

implementation 8

inception phase of a UP project 7

P

phases of a UP project 7

R

requirements analysis 8

S

scope of systems 16

software requirements 25

T

testing 8

timeboxing 7

transition phase of a UP project 7

U

Unified Process (UP) 7

use cases 5, 17

user requirements 19

V

Volere template 19

functional requirements 19

non-functional requirements 20

product constraints 19